AI-Infused Development: Lessons Learned and Patterns from Esri Engineers

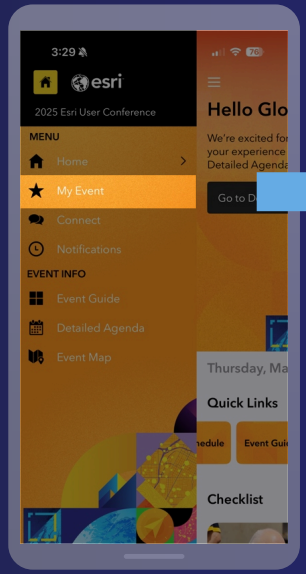
Raul Jimenez Ortega, Sascha Brunner

ESRI EUROPEAN DEVELOPER & TECHNOLOGY SUMMIT 2026

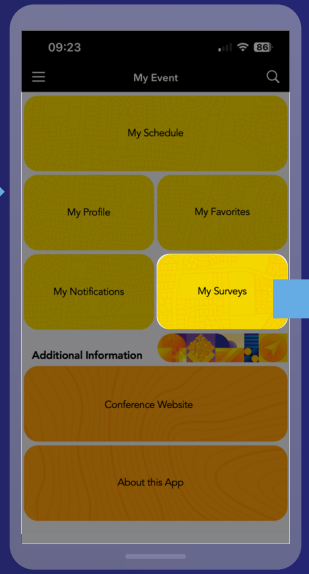
Copyright © 2026 Esri. All rights reserved.

Please share your feedback in the app

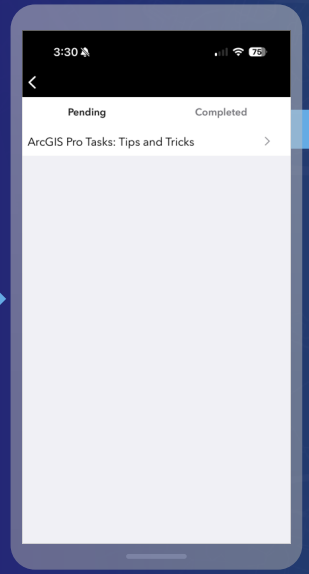
Select “My Event” in the event app main menu



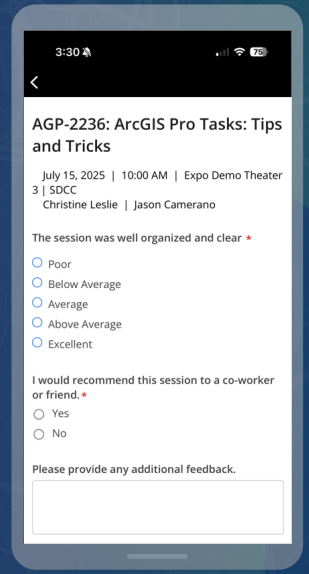
Select “My Surveys”



Select the desired survey

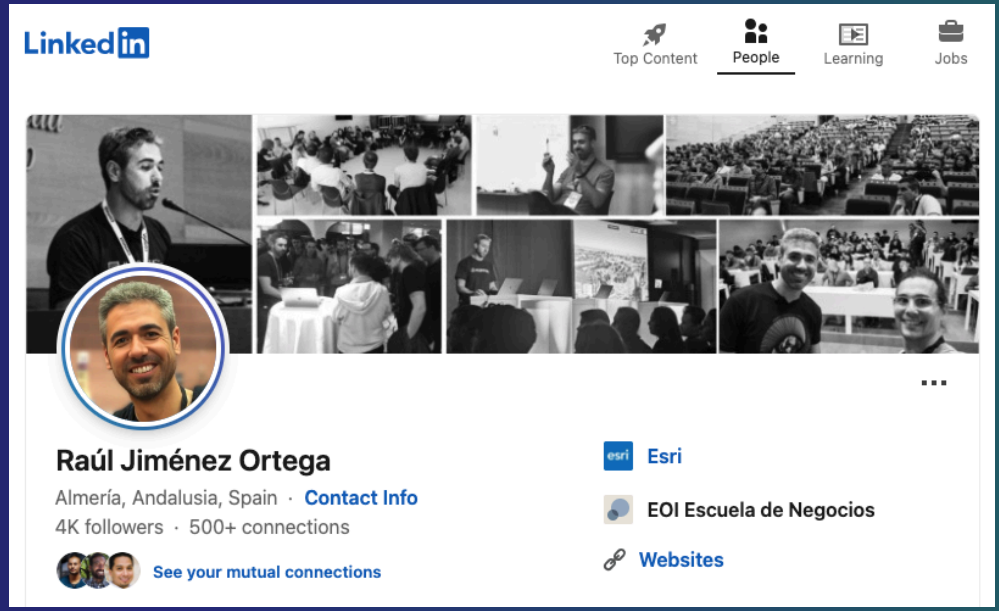


Complete the survey



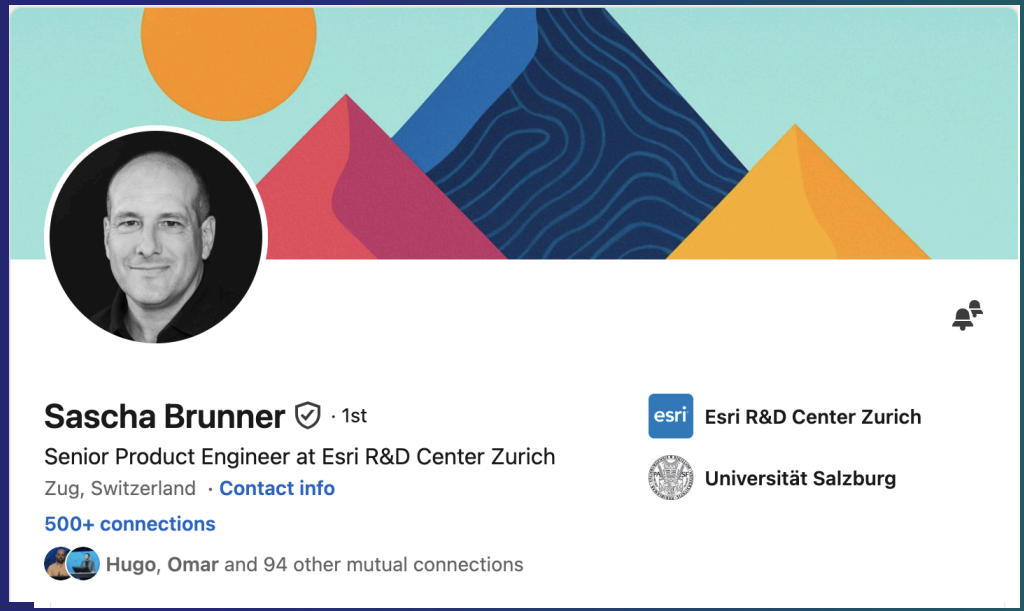
Raul Jimenez Ortega

- 👩‍🎓 Computer Science Degree
- 👥 Developer Experience Team
- 🥷 Principal Product Engineer
- 🎯 Developer Relations
- 🗣️ JavaScript, Python, REST, …
- 🤖 AI Since 2023



Sascha Brunner

- 👩‍🎓 Computer Science & GIS
- 👥 Web Team
- 🥷 Senior Product Engineer
- 🎯 ArcGIS Maps SDK for JS
- 🗣️ JavaScript, TS, 3D, ...
- 🤖 AI Since 2025 (Opus 4.5)



Agenda

- Setting the stage
- AI across the development cycle
- Risks and limitations
- Tips to mitigate
- Wrap-up
- Questions

Disclaimer

What to expect from this session

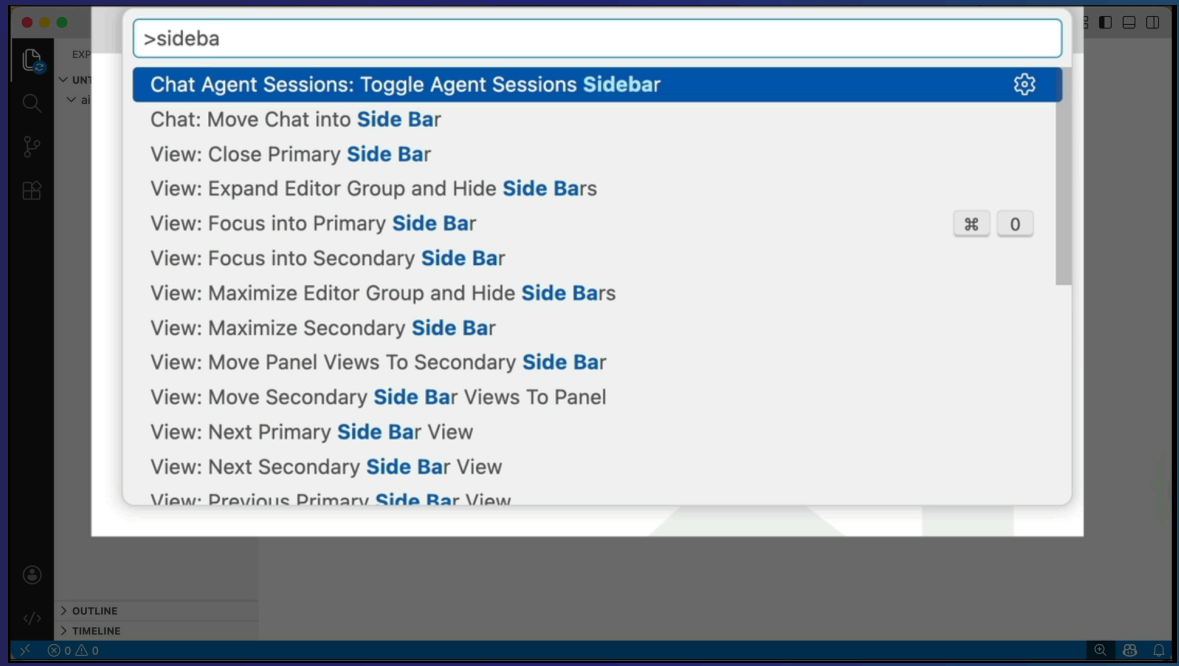
- **Audience**: all SDKs, all roles: builders, dev, devOps, PMs, tech leads, IT professionals
- **Sources:** lessons from Esri engineers and the broader developer community
- **Format:** content-heavy by design
  - "*I've made this letter longer than usual because I don't have time to write a shorter one*" Blaise Pascal, 1657
- **Topic:** About how to use AI to assist you in app development (not to build agentic apps)
- **Built with:** the tools we're about to discuss

Setting the stage

Raul Jimenez Ortega

Vibe Coding: GitHub Copilot Coding Agent from VS Code

How it feels? (For those new to coding agents in the room)

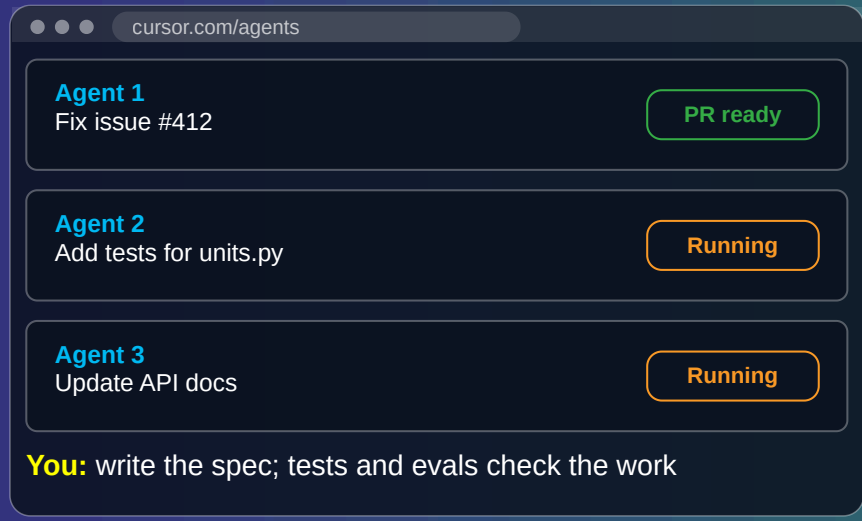


Python script to publish file-based datasets to ArcGIS Online

The Evolution of AI Coding

From suggesting the next line to delegating whole tasks

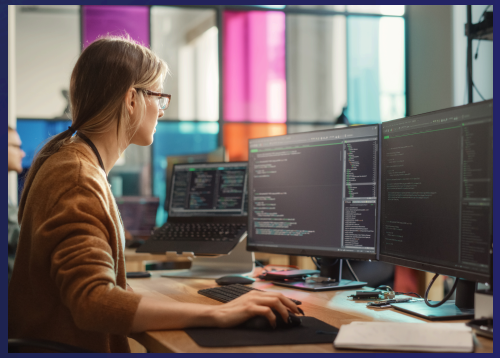
- **2021 Tab completion**
  GitHub Copilot preview, Jun 2021
- **2023 Chat assistants**
  ChatGPT (Nov 2022), GPT-4 (Mar 2023)
- **2024 AI-native IDEs**
  Cursor, Claude 3.5 Sonnet, MCP
- **2025 Coding agents**
  Claude Code (Feb), Codex CLI (Apr)
- **2025–26 Cloud agents**
  Cursor cloud agents, Codex cloud
- **Next Harness engineering**
  “Building the thing that builds the thing”



Agents work for hours, then return tested PRs

Coding is no longer limited to the laptop

What’s happening...

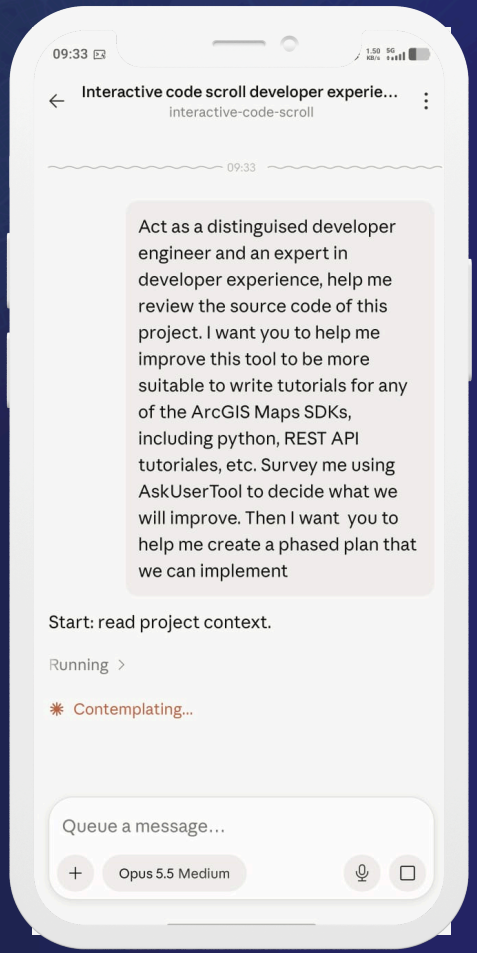


~2000
Fixed workplace
Desk-bound coding

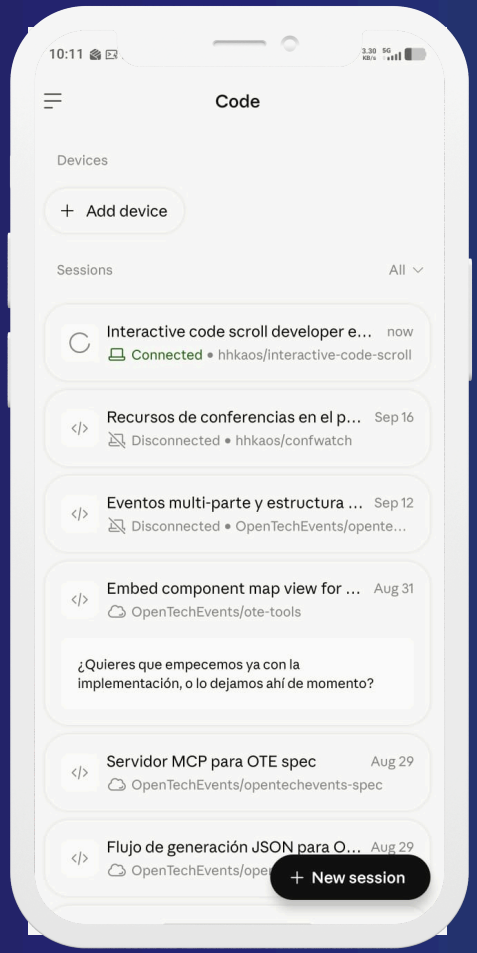
~2010
Portable workplace
Bound to the laptop

~2025
Agentic workplace
Boundaries dissapear

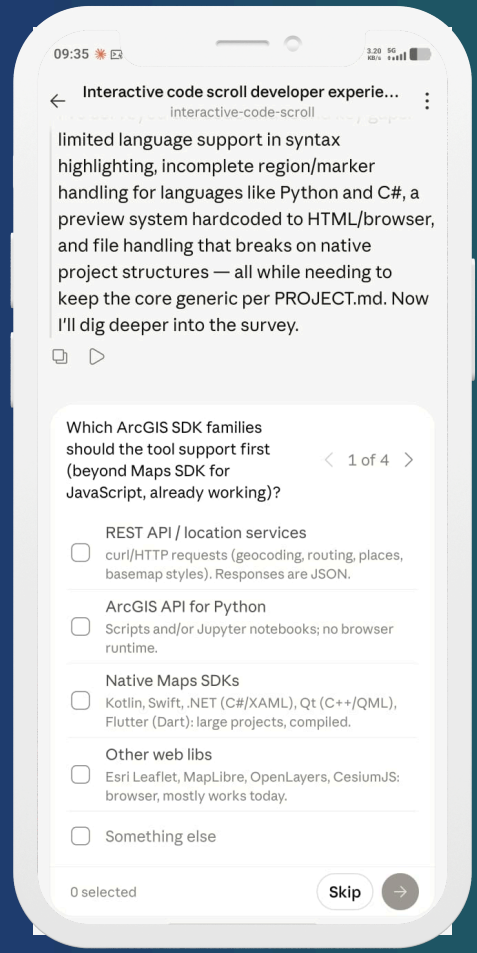
Remote control



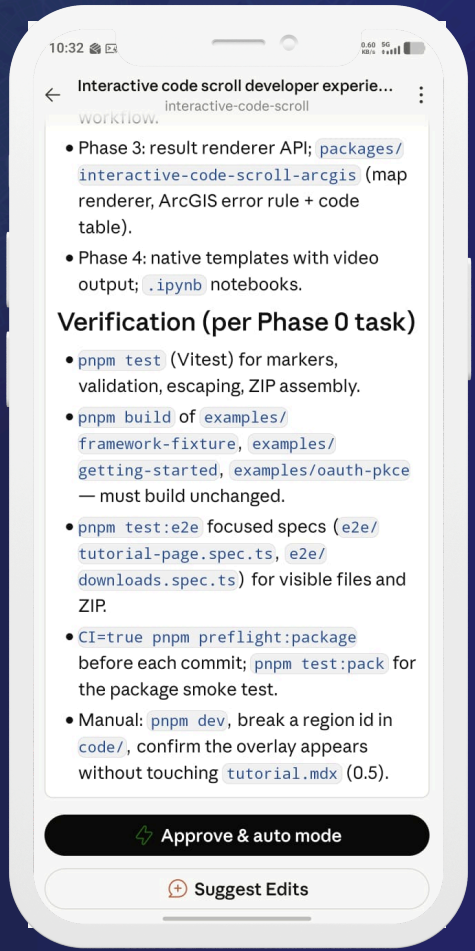
Concurrent sessions



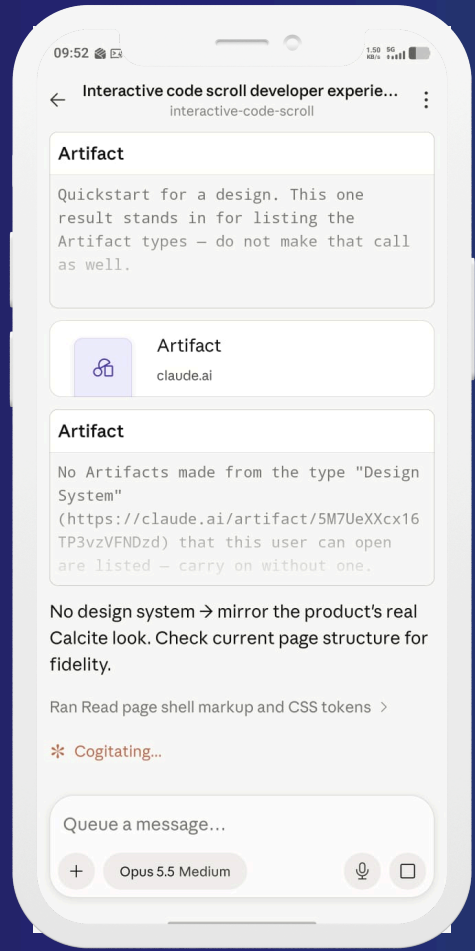
Surveying you



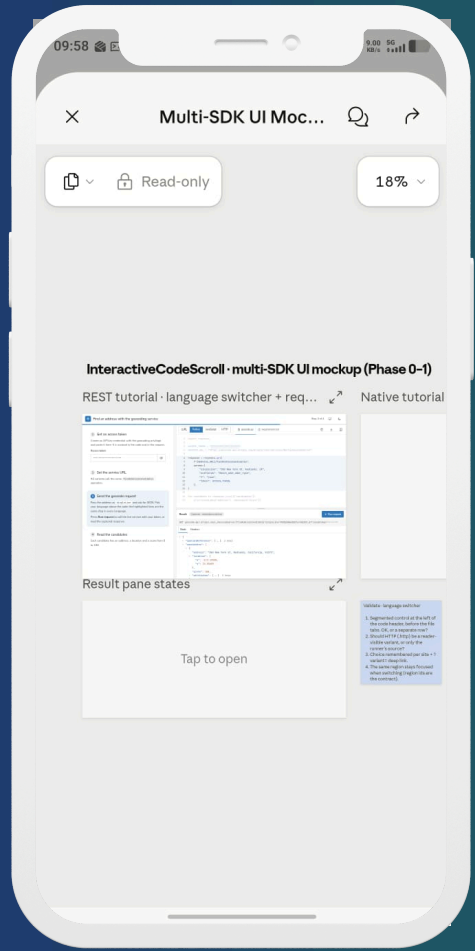
Planning



Creating artifacts



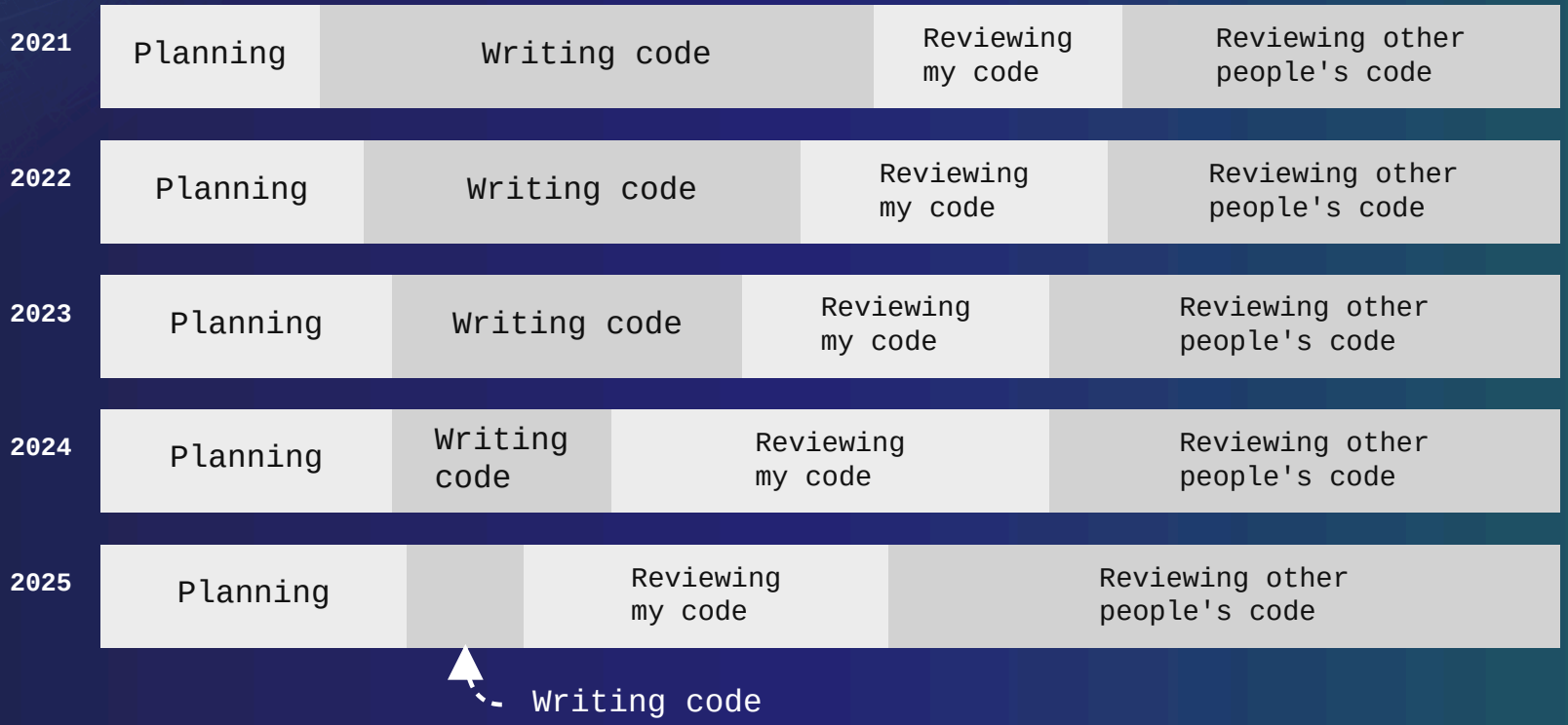
Interactive mockups



From: Louis Knight-Webb, Vibe Kanban Experience (not measured industry data)

Software Engineering Is Becoming Plan and Review

Shift in Software Engineering Ratios



Glossary: anatomy of an agent

AGENT = Model + Harness + Goal Goal: *“Fix the failing buffer test and open a pull request”*

HARNESS Tools: read_file edit_file run_tests git Permissions: ask before git push

Context management: loads AGENTS.md, saves memory, compacts long history

CONTEXT

PROMPT
“Fix the failing buffer test. Don’t change the public API.”

```
+ AGENTS.md
  memory.md
  src/geometry/buffer.ts
  test_output.log
```

AI MODEL

“The test expects meters but gets feet. I’ll convert units before buffering.”

EXECUTION LOOP

1 Read buffer.ts + test

2 Edit buffer.ts

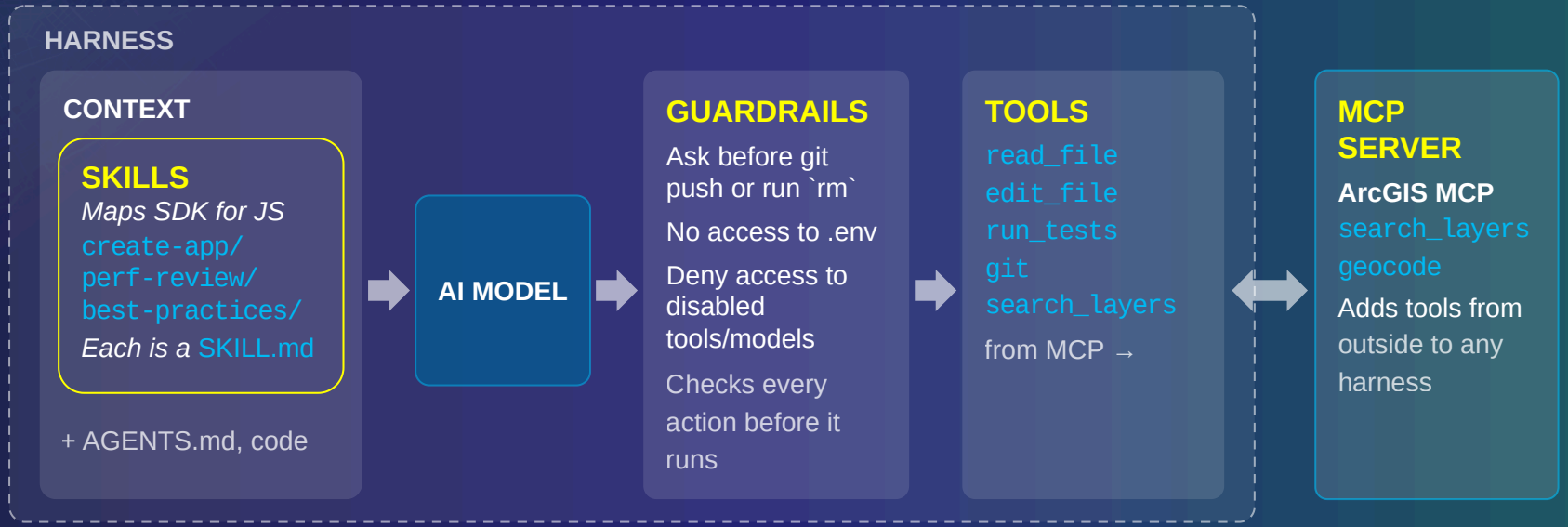
3 Run tests: 1 fails

4 Fix edge case, re-run

Done when all tests pass

Learn more: Harnesses in AI: A Deep Dive

Glossary: what the agent works with



EVALS **For those building agents or testing SKILLs:** run the agent on 50 sample tasks and score the results (46/50 pass)

Just using AI to write deterministic code? Your unit tests already do this job.

A Wide Variety of Options

	CLOSED The vendor builds and runs it	OPEN Weights or code are public
Models The brain	Claude, GPT, Gemini, Grok **Access:** via API or subscription	Qwen, DeepSeek, GLM, Kimi, gpt-oss, Mistral **Sovereignty:** run them on your own servers
Harnesses The software around it	Claude Code, Cursor, GitHub Copilot, Kiro **Convenience:** vendor owns the roadmap	OpenCode, Codex CLI, Gemini CLI, Cline, Pi, Aider **Control:** read, fork and extend the code

Harness and model are separate choices

Uses models from many vendors

Mainly its own vendor's models

Examples only; the list changes every few months. Open = public weights (models) or open-source code (harnesses).

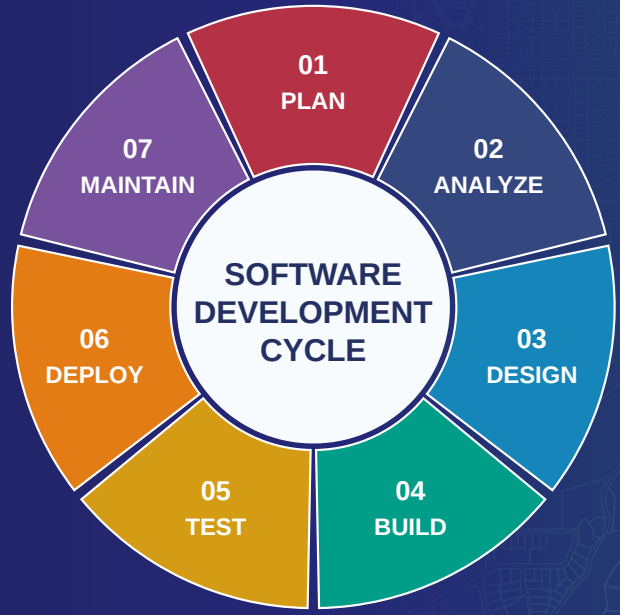
AI across the development cycle

Sascha Brunner

CYCLE / OVERVIEW

AI support at each development stage

At each stage, choose who leads and who helps.



Illustrative SDLC cycle · stages overlap and repeat

PLAN / FROM OBSERVATION

Planning: from observation to PRD

PRD = Product Requirements Document

- Observe
- Agree
- Test

SHORT PRD / ILLUSTRATIVE

User task	Select a feature
Success	Details appear
Check	Test + screenshot
Open	Who can access it?

PLAN / PRD TO SDD

Spec-driven development

Agree on testable requirements before AI writes code.

APPROVED SCENARIOS

Agree → approve → build

ILLUSTRATIVE SCENARIO

```
Given an accessible Web Scene
When I select a feature
Then its details appear
```

Small phases: build → test → review → repeat.

ARCHITECTURE OPTIONS

Faster prototypes. More options.

Compare approaches and agree together.

Codebase exploration

Give context: SDK version, Online/Enterprise, schema and constraints.

- User tasks
- Code
- Project rules

SHARED REPOSITORY / EXAMPLE

AGENTS.md	Project instructions
docs/decisions/	Why we chose this approach
examples/	Working reference patterns
skills/	Task-specific guidance

From bug report to actionable issue

Record the facts before asking an agent to fix it.

- Reproduce
- Describe
- Define success

EXAMPLE ISSUE

Problem	Details stay after clearing selection
Expected	Details disappear
Evidence	Steps + screenshot
Test	Select a feature, then clear it

DESIGN / UI OPTIONS

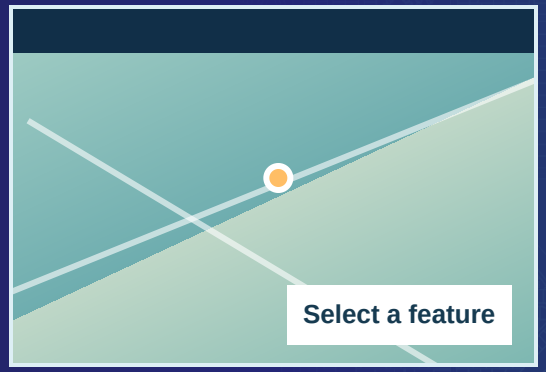
UI prototyping: compare alternatives

Illustrative alternatives based on a team's prototyping practice.

A · MINIMAL

Fast path to the main task

B · GUIDED



More cues for a first visit

C · POWER USER

More control, more complexity

Ask AI: “What assumptions could make each option fail?”
Choose by observed behavior.

DESIGN / MODERNIZATION

Migration: update or rebuild?

Web GIS SDK engineers reported an AI-assisted Vitest migration.

ILLUSTRATIVE DECISION

App on outdated technology

Revisit deferred migration work.

↗

PATH A

Migrate in place

Update the existing code.

↘

PATH B

Rebuild from scratch

Create a new codebase.

When rebuilding, use the old codebase to find edge cases. Improve the new app.

Scene authoring with a browser agent

An agent uses Scene Viewer to create a 3D scene from an existing Web Map.

01 Author

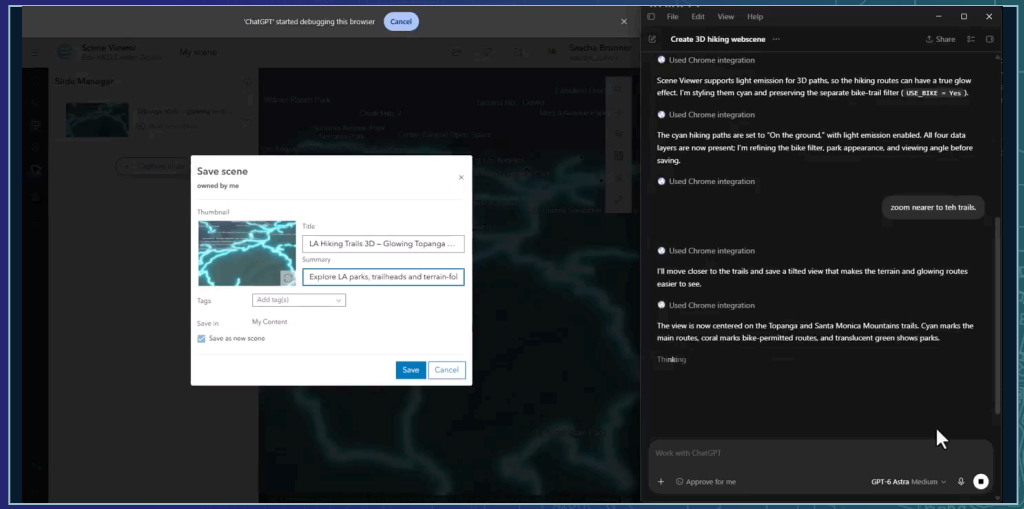
Choose data, styling, pop-ups and the opening view.

02 Save

Check the Web Scene item and its access.

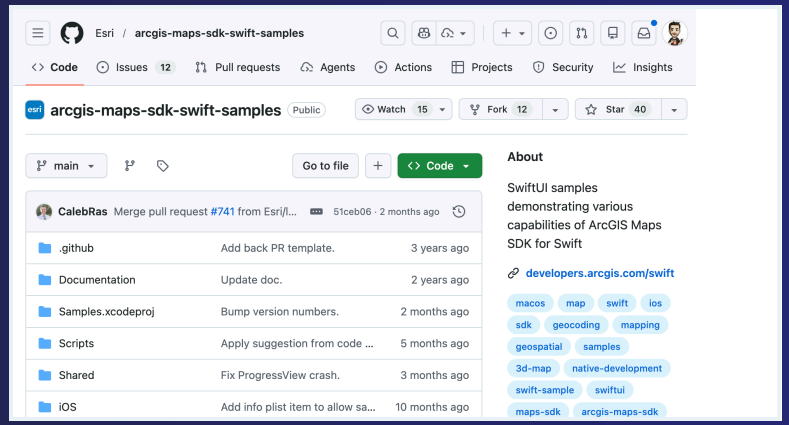
03 Verify

Check pop-ups, layers, sharing and attribution.



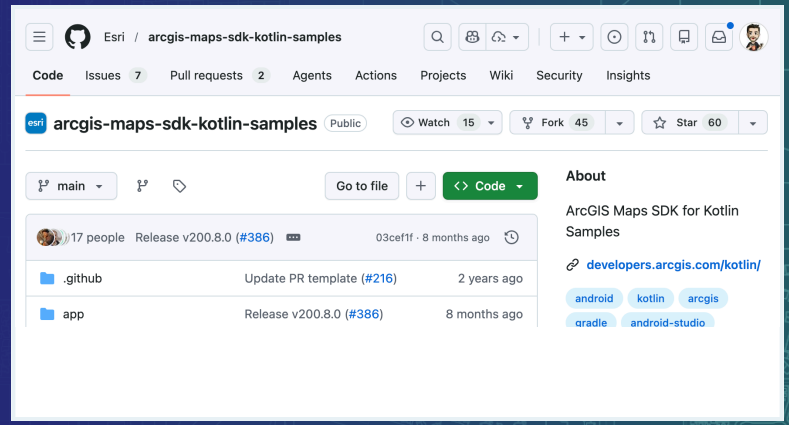
Cross-SDK sample adaptation

A working source behavior helps explore an equivalent in another language.



Swift samples · source behavior

→



Kotlin samples · target pattern

Adapt the pattern, then build, run and review the target result.

BUILD / CODE QUALITY

Code quality: lint and type errors

Business Analyst used agents for lint and TypeScript cleanup.

01 / SMALL TASKS

Isolate each task

One worktree + clear scope

→

02 / AI HELP

Draft the fixes

Separate parts of the code

→

03 / CHECK

Run checks and review

Types, lint and app behavior

Suggested workflow

Fewer errors do not prove the app works.

TEST / PERFORMANCE

Rendering performance experiments

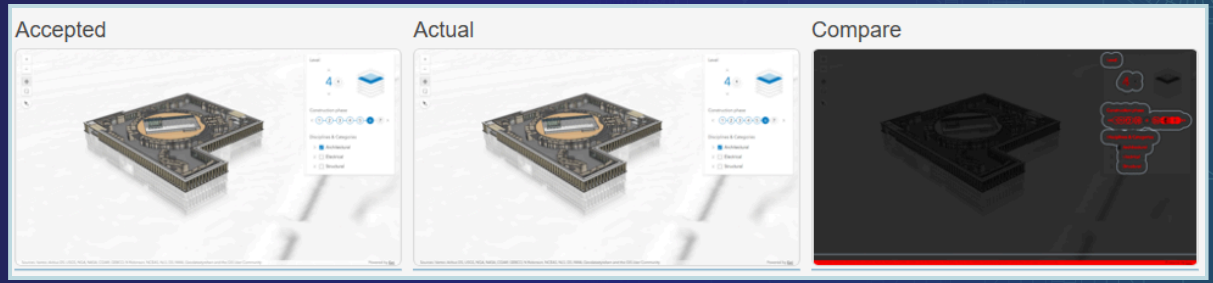
Meet the visual target within CPU, GPU and memory budgets.

AI drafts variants → **Run the same scene** → **Compare visual + resources**

APPROACH	VISUAL TARGET	CPU	GPU	MEMORY	DECISION
A · Full detail	MEETS	OK	OVER	OVER	Revise
B · Fewer details	MISSES	OK	OK	OK	Revise
C · Adaptive detail	MEETS	OK	OK	OK	SHORTLIST

Illustrative runs · fixed scene, camera, data and device · shortlist still needs validation

Visual testing: diagnose differences



UI / COMPONENTS	Sample code, Calcite alert or component change?
SCENE / MAP PIXELS	SDK rendering or view-state change?
DATA	Missing layers or changed layer appearance?
AI-ASSISTED TRIAGE	Screenshots + logs + code history → likely cause → fix → rerun

Testing: wait for the view state

Copilot helped find dependencies on fixed delays.

BEFORE / FIXED DELAY

Wait a fixed time

Start ..... Check

Elapsed time does not prove readiness.

AFTER / VIEW STATE

Wait for the required state

Start → View state → Check

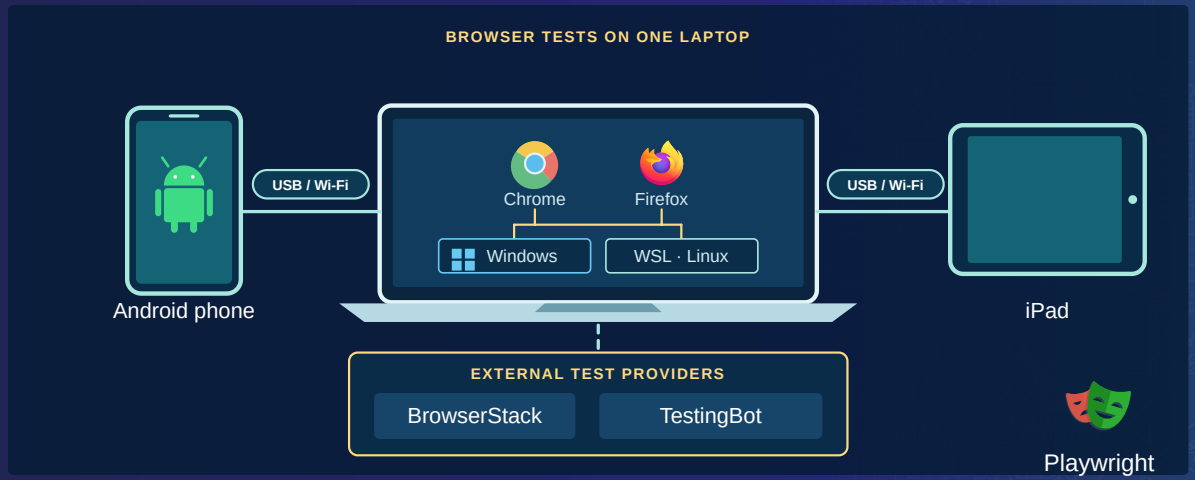
Tests now use `view.updating`.

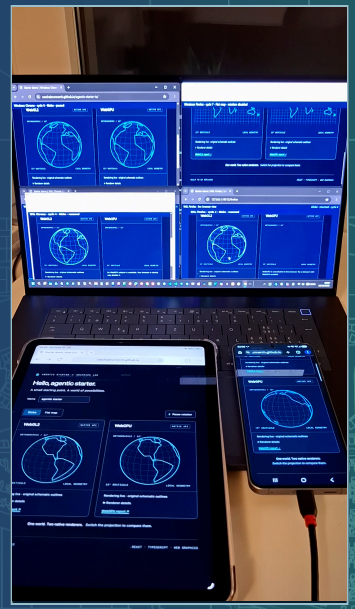
The ready state depends on the test.

Understand the delay before removing it.

Browser and device testing

Record where the browser runs and what the result proves.





Emulation ≠ a real device

Browser + OS + device + result

Graphics work: verify the renderer too.

TEST + DEPLOY / REPEATABILITY

Handling non-deterministic AI

AI proposals can vary. Make the surrounding workflow repeatable.

01 / INPUTS

Make requirements explicit

Stable task, data and environment

02 / OPERATIONS

Use scripts and tools

Repeat the agreed steps

03 / CHECKS

Test agreed behavior

Automated checks with clear criteria

04 / REVIEW

AI review → fix → human review

Scenarios + project rules + full diff

DEPLOY

AI advice cannot replace release gates.

Reviewed scripts · required checks · approval rules

Documentation: write and validate

Write docs with the change, then check the guide against the running app.

A mismatch can mean a mistake in the guide or the software.

Context management and handoffs

An agreed decision can disappear from a long conversation.

WITHOUT A HANDOVER / ILLUSTRATIVE

Agreed earlier

Keep the access rules.

↓ Conversation shortened

Later continuation

Agent changes the access rules.

HANDOVER / ILLUSTRATIVE

Goal	Feature details appear
Decision	Keep access rules
Done	Details UI updated
Checks	Type check passed
Next	Test feature selection

Review → correct → save → share.

Keep verified lessons in project rules, examples or checks.

BEHIND THE DECK

Creating this deck with AI and Slidev

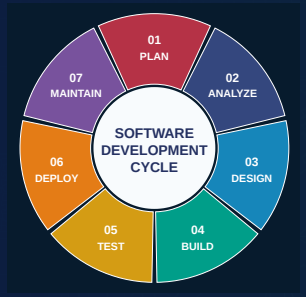
Prompt an agent in VS Code; Slidev renders editable Markdown, HTML and SVG.

ILLUSTRATIVE PROMPT IN VS CODE

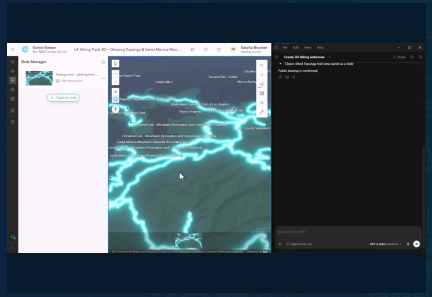
“Add the SDLC SVG and my Scene Viewer recording to this editable Slidev deck.”

- `slides.md`
- `theme.css`
- `assets/diagrams/*.svg`
- `public/media/*.mp4`

PREVIEW IN SLIDEV



Prompted SVG



Captured video frame

Prompt → edit → preview → capture → refine.

HANDOVER

Getting started: one small task

Use these five steps on your next development task.

1

Choose a task

One useful change

2

Define success

Expected behavior

3

Give context

Versions, schema, constraints

4

Run checks

Tests and running app

5

Review results

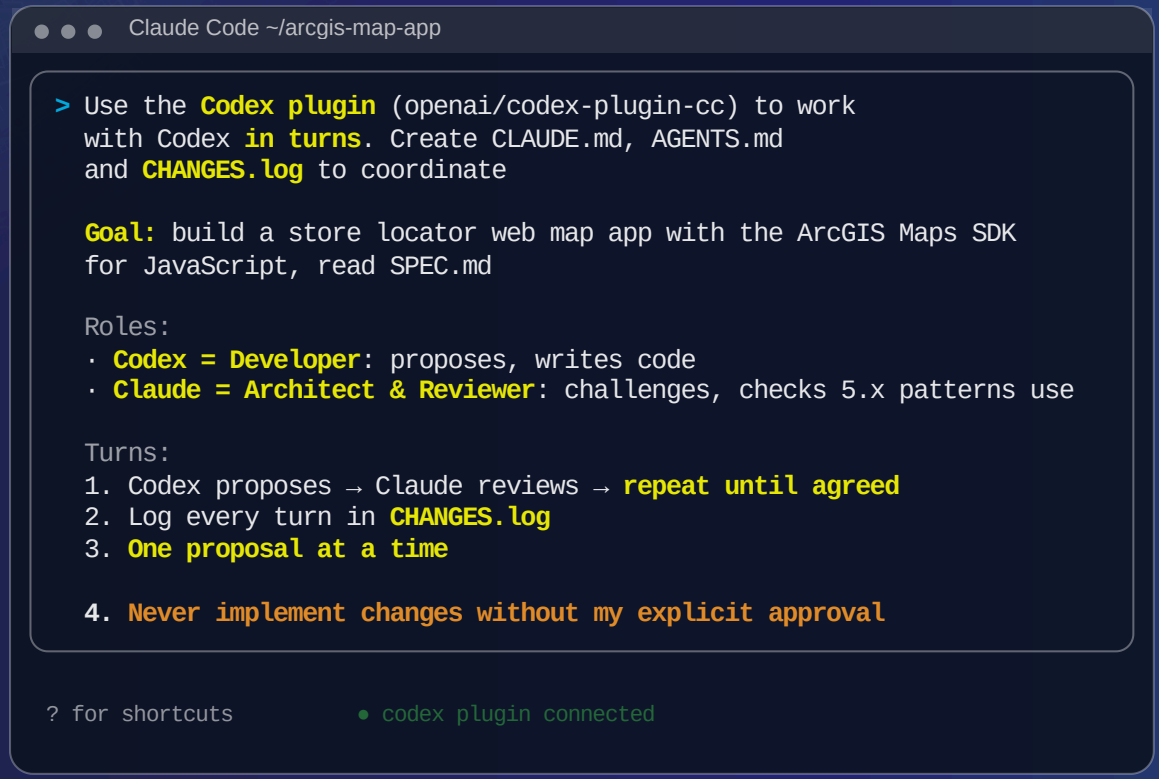
Save verified lessons

MY PRACTICAL STARTING POINT

github.com/SaschaBrunnerCH/agentic-starter-ts ↗

Put several models to work together doing handoffs

1. The prompt: what to type



1. **Tool and shared files**
   Claude can call Codex
2. **Goal**
   What to build
3. **Roles**
   Who proposes, who reviews
4. **Turns and hand-off**
   Every turn goes through CHANGES.log
5. **Human in the loop**
   Agents stop until you approve

Put several models to work together doing handoffs

2. What happens next: turns, hand-offs and your approval

CLAUDE CODE
Architect & Reviewer
Checks every proposal against 5.x patterns

CHANGES.log

```
## CODEX — PROPOSAL #1
- Done: phase 1, map + Search widget

## CLAUDE — REVIEW #1
- Verdict: CHANGES_REQUESTED
- Why: Search widget is 4.x; use arcgis-search

## CODEX — REVISION #1
- Fix: widget → arcgis-search

## CLAUDE — REVIEW #2
- Verdict: OK. Waiting for user

## YOU — APPROVED
- Go ahead with phase 2

## CODEX — PROPOSAL #2
- Working: phase 2, store list + directions…
```

CODEX
Developer
Proposes and writes the code

YOU human in the loop — Approved

Real example: it worked, and still got it slower

René Rubalcava built an ArcGIS app with Claude and Codex, then read the diff

01 · FIX

Codex fixes a bug

- Widens the layer's **outFields** so the sidebar can list addresses

02 · SIDE EFFECT

Panning gets slow

- Every feature drawn on the map now carries **extra fields**
- The app still "works"

03 · AI REVIEW

Claude reviews Codex

- Finds **real issues**: UI flicker, orphaned watch handles
- Some findings **refuted** against SDK typings

04 · HUMAN

René reads the diff

- Queries the feature layer instead
- **One-line fix**, panning is instant again

“These tools are good at writing code fast. They are not a substitute for reading it.”

Source: René Rubalcava, “ArcGIS Online, Maps SDK for JavaScript & AI”, odoe.net/blog/ago-to-comps

Risks and limitations

Raul Jimenez Ortega

Powerful, but not limitless

Before we move on: the risks and limits we all need to know

Security & privacy
Secrets · MCPs · prompts injection · malicious pkgs

Code quality
Hidden bugs · missing use cases ·over-engineer

Trained on public, older code
Cutoff · deprecated APIs

Still makes mistakes
Hallucinated APIs

Not deterministic
Varies run to run

Limited context window
Compaction · token limits

Doesn't know our team rules
Conventions · history

ArcGIS pitfalls & gaps
Arcade · ArcPy · Calcite

The elephant in the room: AI comes with a real cost to us

It feel productive (for some, addictive), but can quietly drain our skills, focus and joy.

"We used to call it an engineer, now it is like a reviewer."

Siddhant Khare, software engineer, on burning out from AI-driven work[1]

AI fatigue

Re-prompting, micro-decisions and context switching drain focus

Tip: Cap loops at ~3 rounds, then review in a separate pass

Tech debt → cognitive debt

The code works, **but nobody knows why anymore**

Tip: Carefully review; update design docs; code without AI regularly

Juniors skip the foundations

Senior-speed output without the practice that builds seniors

Tip: Ask juniors to explain every line of their PR

Loss of ownership and joy

From craftsperson to full-time reviewer of AI output

Tip: Keep people in design; tag AI commits

FOR TEAMS Measure comprehension and review quality, not lines of code

[1] Business Insider, Feb 2026. Cognitive debt: M.-A. Storey, "From Technical Debt to Cognitive and Intent Debt", 2026

Video: The struggle is real

Tips to mitigate

Time to take your phones 😉

Tips to mitigate: Security and privacy issues

Start with the basics, add layers as agents do more

STEP 1

Basic hygiene

- Use **synthetic** or anonymized data only
- **Least privilege**: fine-grained tokens, no full GitHub CLI
- **Deny list** for sensitive paths
- **Human in the loop**: review security code, audit agent actions

STEP 2

Supply chain

- **Verify package names** before installing
- Commit and use **lockfiles**
- Run **npm audit** / **pip audit**
- **Lock down pnpm** config (pnpm-workspace.yaml)
- Use **mock servers**, not real services

STEP 3

Isolation and control

- Run agents in an **isolated sandbox**: dev container or ephemeral cloud VM
- **Dependency firewall**, registry proxy or local CLI defender
- Customize an **open-source harness**
- Use **open-weight models**
- Add **evals** for agent behavior

Tips to mitigate: technical limitations

Know what limits the agent, then give it the context it's missing

1 · THE AGENT

Limited memory (fills up with each prompt)

HOW TO MITIGATE

- **Handover notes** and fresh sessions
- **Token savers** (Ponytail, Caveman)
- **Specs and tests** keep results repeatable

2 · YOUR PROJECT & TEAM

Doesn't know your code or decisions

HOW TO MITIGATE

- **AGENTS.md** with gotchas, pitfalls, arquitectural doc …
- **Hierarchical context**: shared across projects, specific down to folder or file type
- **Team knowledge base** via your own MCP server

3 · ARCGIS

Learned from mixed public samples

HOW TO MITIGATE

- State **product, version, license, user type and auth**
- **Known pitfalls** (+next slide)

COMING SOON · **ArcGIS Agentic Developer Toolkit**
Skills + MCP for coding agents

ArcGIS: where AI makes mistakes

Recurring pitfalls from years of public samples, across versions and products

ARCADE

Uses variables from the wrong profile

Fix: Name the profile: pop-up, label, field calc

ARCPY

Deprecated calls, wrong version or license

Fix: State the Pro version and license level

JS MAPS SDK

Mixes widgets with components

Fix: Pin the SDK version; link the component docs

CALCITE

Overrides the design system with custom CSS

Fix: Slots and CSS variables, no shadow DOM hacks

LAYER VIEWS

Assumes every field is already loaded

Fix: Query the layer for extra fields

Write & keep updating context (AGENTS.md) and skills so it get better on each session

Wrap up

Raul Jimenez Ortega

AI is here to stay

Habits that keep AI responsible, effective and secure

ORGANIZATION what you allow

Set policy before rollout
Approved tools, use cases and data handling

Vet the vendor
Data retention, IP ownership, long-term alignment

Harden in layers
Least privilege, sandboxes, dependency firewall

Stay free to switch
Pick model and harness separately

TEAM how you work together

Share context as code
Common AGENTS.md, skills and hierarchical context

AI assists, humans approve
Let AI review AI; a person decides and merges

Grow juniors on purpose
They explain every line of their PR

Measure understanding
Comprehension and review quality, not lines of code

YOU the developer

Give it your context
Spec, AGENTS.md, product and version up front

Work in small steps
Phase the work, stay in the loop, cap retries at ~3

Read the diff, test every phase
Verify API claims; add a test per bug

Keep your own skills sharp
Write down intent; code without AI regularly

BOTTOM LINE AI writes code fast. Understanding, owning and securing it is still on us.

“🌍 AI is reshaping how we work - and it’s blurring the lines between professions.”

Vinay Viswambharan,
Principal Product Manager at Esri

Whether we like it or not, use it or not… this is already happening

Docs & Resources

Habits that keep AI responsible, effective and secure at every level

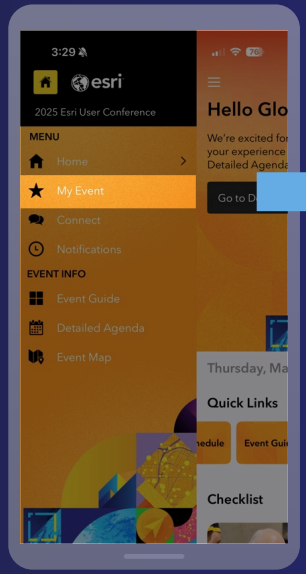
- JavaScript SDK
  - AI Assisted Development
  - Intro to building agentic mapping applications
- Experience builder
  - Building Custom Experience Builder Widgets with an Agent Skill
- Python API
  - Introduction to AI-Powered services in ArcGIS Online
- MCPs
  - MCP in ArcGIS Enterprise
  - MCP for ArcGIS Location Services
  - MCP in ArcGIS Online (coming soon)
  - Geoportal 3.1.0 added a Model Context Protocol (MCP) API to its STAC configuration
- **ArcGIS Agentic Developer Toolkit (coming soon)**
- Agentic AI > **Architecture center > Agentic AI**

Esri DevTech Summit 2026
(March, Palm Springs)

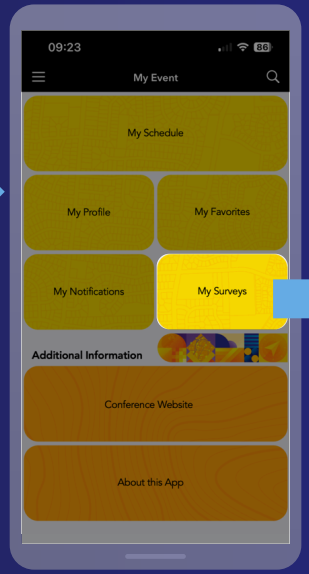
Questions

Please share your feedback in the app

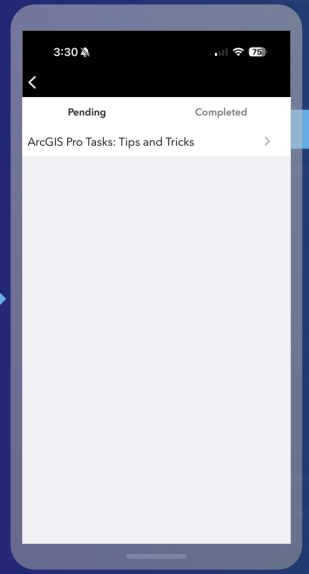
Select “My Event” in the event app main menu



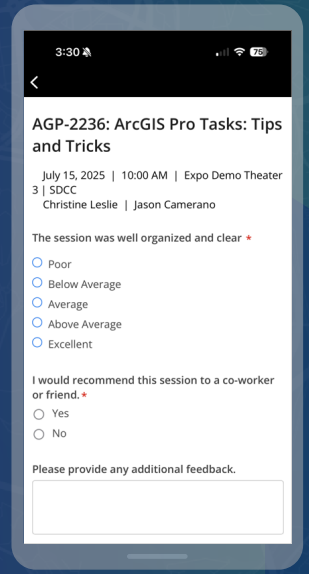
Select “My Surveys”



Select the desired survey



Complete the survey



Connect with us on social

Join the conversation using #EUDevTech2026

- linkedin.com/showcase/esri-developers
- linkedin.com/groups/1949564/
- x.com/EsriDevs
- x.com/EsriDevEvents
- youtube.com/@EsriDevs
- links.esri.com/DevVideos
- github.com/Esri
- github.com/EsriDevEvents
- links.esri.com/EsriDevCommunity

esri | THE SCIENCE OF WHERE®

Copyright © 2026 Esri. All rights reserved.
Portions of this work may include Adobe Stock images used under license. Copyright © 2026 Adobe Inc. All rights reserved.